ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 2 — MICROSERVICES EDGE CASE TESTING

Sagum, Patrick Ruiz; Henson, Princess Terana Caram Rasonable;
Gargarita, Trisha Faith Casiano; Mogat, Ela Mae Trojillo;
Tibo-oc, Paul Felippe Gelle
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise tested three microservices — student-service (port 8001), course-service (port 8002), and enrollment-service (port 8003) — against a 27-step sequential test suite covering HTTP status codes 200, 201, 400, 404, 409, 503, and 504. All tests were executed using curl -i to capture both response headers and JSON bodies in deterministic order, ensuring each step had the required data before proceeding to the next. The key finding is that the enrollment-service is the only service that returns 503 and 504 errors because it is the only service with external HTTP dependencies on student-service and course-service. All 27 test steps were confirmed to return the correct HTTP status codes and consistent ERROR_CODE strings across all services.

I. INTRODUCTION

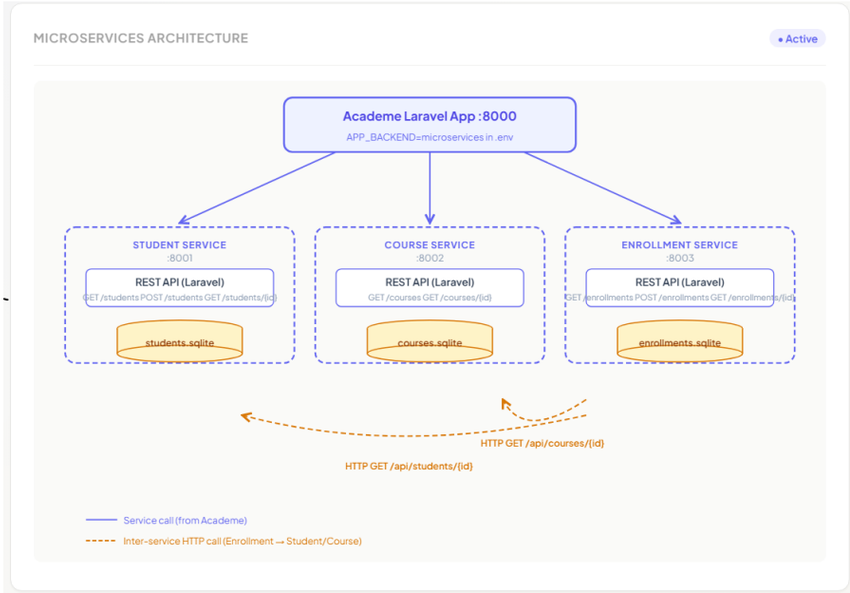
Edge case testing verifies that a system handles abnormal inputs and failure conditions correctly. Testing only the happy path — valid inputs producing expected outputs — is insufficient for production systems. Real-world APIs receive malformed requests, reference nonexistent resources, and depend on services that can fail or respond slowly. Each of these scenarios must return a meaningful HTTP status code and a structured error response that clients can parse and handle programmatically.

HTTP status codes communicate the outcome of every API request. 2xx codes indicate success: 200 OK for reads and updates, 201 Created for new resources. 4xx codes indicate client errors: 400 Bad Request for invalid input, 404 Not Found for missing resources, 409 Conflict for duplicate entries. 5xx codes indicate server-side failures: 503 Service Unavailable when a dependency is offline, 504 Gateway Timeout when a dependency is too slow. Microservices introduce 503 and 504 scenarios that monolithic systems never face — a monolith has no external HTTP dependencies, making connection failures and timeouts architecturally impossible.

The test suite in this report is ordered sequentially so that each step builds on the data created by previous steps. This deterministic approach ensures reproducible results and eliminates test failures caused by missing or inconsistent data. A fresh database seed is performed before every full run to establish a known baseline of three students and three courses.

II. METHODOLOGY / IMPLEMENTATION

All three services were started on separate terminal windows before testing began. The enrollment-service depends on student-service and course-service for cross-service validation, so all three must be running simultaneously. A fourth terminal was used exclusively for curl commands. The test suite follows a fixed sequence of 27 steps covering full CRUD operations, all validation and not-found edge cases, duplicate detection, cross-service failure scenarios, cleanup, and verification.



The following table lists every edge case scenario implemented and tested:

Status	Error Code	Trigger	Service
201	—	Successful creation	All
200	—	Successful read / update / delete	All
400	VALIDATION_ERROR	Missing or invalid input fields	All
404	NOT_FOUND	Resource ID does not exist	All
404	STUDENT_NOT_FOUND	Invalid student_id on enroll	Enrollment
404	COURSE_NOT_FOUND	Invalid course_id on enroll	Enrollment
409	DUPLICATE_EMAIL	Email already registered	Student
409	DUPLICATE_ENROLLMENT	Student already in course	Enrollment
503	SERVICE_UNAVAILABLE	Dependency service offline	Enrollment
504	GATEWAY_TIMEOUT	Dependency response too slow	Enrollment

A. Reset and Seed (Required Before Every Full Run)

Run the following from the project root before starting the test suite. This establishes a clean baseline of three students and three courses with the enrollment table empty.

```
cd lab2/services/student-service && php artisan migrate:fresh --seed
cd ../course-service          && php artisan migrate:fresh --seed
cd ../enrollment-service      && php artisan migrate:fresh
```

B. Start Services

Open four separate terminal windows:

```
# Terminal 1
cd lab2/services/student-service && php artisan serve --port=8001

# Terminal 2
cd lab2/services/course-service  && php artisan serve --port=8002

# Terminal 3
cd lab2/services/enrollment-service && php artisan serve --port=8003
```

```
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ php artisan serve --port=8001

  INFO  Server running on [http://127.0.0.1:8001].

  Press Ctrl+C to stop the server
```

```
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1 (main)
$  cd /c/Users/parus/Desktop/sar-lab1/lab2/services/course-service && php artisan serve --port=8002

  INFO  Server running on [http://127.0.0.1:8002].

  Press Ctrl+C to stop the server
```

```
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ php artisan serve --port=8003

  INFO  Server running on [http://127.0.0.1:8003].

  Press Ctrl+C to stop the server
```

C. Sequential Test Flow

Use Terminal 4 for all curl commands. Execute steps in order — do not skip steps.

Step 1 — GET all students (Expected: HTTP 200)

```
curl -i http://localhost:8001/api/students
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i http://localhost:8001/api/students
HTTP/1.1 200 OK
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:51:35 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":[{"id":1,"name":"Juan dela Cruz","email":"juan@example.com","created_at":"2026-03-15T03:46:17.000000Z","updated_at":"2026-03-15T03:46:17.000000
Z"},{"id":2,"name":"Maria Santos","email":"maria@example.com","created_at":"2026-03-15T03:46:17.000000Z","updated_at":"2026-03-15T03:46:17.000000Z"},{"
id":3,"name":"Pedro Reyes","email":"pedro@example.com","created_at":"2026-03-15T03:46:17.000000Z","updated_at":"2026-03-15T03:46:17.000000Z"}],"message
":"success"}
```

Step 2 — GET all courses (Expected: HTTP 200)

```
curl -i http://localhost:8002/api/courses
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i http://localhost:8002/api/courses
HTTP/1.1 200 OK
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:52:01 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":[{"id":1,"title":"Introduction to Programming","description":"Learn the basics of programming concepts and logic.","created_at":"2026-03-15T03:
46:19.000000Z","updated_at":"2026-03-15T03:46:19.000000Z"},{"id":2,"title":"Web Development","description":"Build modern web applications using current
 technologies.","created_at":"2026-03-15T03:46:19.000000Z","updated_at":"2026-03-15T03:46:19.000000Z"},{"id":3,"title":"Database Systems","description"
:"Understand database design, SQL, and data management.","created_at":"2026-03-15T03:46:19.000000Z","updated_at":"2026-03-15T03:46:19.000000Z"}],"messa
ge":"success"}
```

Step 3 — POST create student (Expected: HTTP 201)

```
curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Sequential Student","email":"seq.student@example.com"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Sequential Student","email":"seq.student@example.com"}'
HTTP/1.1 201 Created
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:52:23 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"data":{"name":"Sequential Student","email":"seq.student@example.com","updated_at":"2026-03-15T03:52:23.000000Z","created_at":"2026-03-15T03:52:23.000
000Z","id":4},"message":"created"}
```

Step 4 — GET student by ID (Expected: HTTP 200)

```
curl -i http://localhost:8001/api/students/4
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i http://localhost:8001/api/students/4
HTTP/1.1 200 OK
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:52:41 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":{"id":4,"name":"Sequential Student","email":"seq.student@example.com","created_at":"2026-03-15T03:52:23.000000Z","updated_at":"2026-03-15T03:52
:23.000000Z"},"message":"success"}
```

Step 5 — PUT update student (Expected: HTTP 200)

```
curl -i -X PUT http://localhost:8001/api/students/4 \
  -H "Content-Type: application/json" \
  -d '{"name":"Sequential Student Updated","email":"seq.updated@example.com"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X PUT http://localhost:8001/api/students/4 \
  -H "Content-Type: application/json" \
  -d '{"name":"Sequential Student Updated","email":"seq.student.updated@example.com"}'
HTTP/1.1 200 OK
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:53:07 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"data":{"id":4,"name":"Sequential Student Updated","email":"seq.student.updated@example.com","created_at":"2026-03-15T03:52:23.000000Z","updated_at":"
2026-03-15T03:53:07.000000Z"},"message":"updated"}
```

Step 6 — POST create course (Expected: HTTP 201)

```
curl -i -X POST http://localhost:8002/api/courses \
  -H "Content-Type: application/json" \
  -d '{"title":"Sequential Course","description":"Course for ordered testing"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i -X POST http://localhost:8002/api/courses \
  -H "Content-Type: application/json" \
  -d '{"title":"Sequential Course","description":"Course used for ordered testing"}'
HTTP/1.1 201 Created
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:53:25 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":{"title":"Sequential Course","description":"Course used for ordered testing","updated_at":"2026-03-15T03:53:25.000000Z","created_at":"2026-03-1
5T03:53:25.000000Z","id":4},"message":"created"}
```

Step 7 — GET course by ID (Expected: HTTP 200)

```
curl -i http://localhost:8002/api/courses/4
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i http://localhost:8002/api/courses/4
HTTP/1.1 200 OK
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:53:44 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"data":{"id":4,"title":"Sequential Course","description":"Course used for ordered testing","created_at":"2026-03-15T03:53:25.000000Z","updated_at":"20
26-03-15T03:53:25.000000Z"},"message":"success"}
```

Step 8 — PUT update course (Expected: HTTP 200)

```
curl -i -X PUT http://localhost:8002/api/courses/4 \
  -H "Content-Type: application/json" \
  -d '{"title":"Sequential Course Updated","description":"Updated for test"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i -X PUT http://localhost:8002/api/courses/4 \
  -H "Content-Type: application/json" \
  -d '{"title":"Sequential Course Updated","description":"Updated for sequential test"}'
HTTP/1.1 200 OK
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:54:08 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 57
Access-Control-Allow-Origin: *

{"data":{"id":4,"title":"Sequential Course Updated","description":"Updated for sequential test","created_at":"2026-03-15T03:53:25.000000Z","updated_at"
:"2026-03-15T03:54:08.000000Z"},"message":"updated"}
```

Step 9 — POST create enrollment (Expected: HTTP 201)

```
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
HTTP/1.1 201 Created
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:54:30 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":{"student_id":4,"course_id":4,"updated_at":"2026-03-15T03:54:30.000000Z","created_at":"2026-03-15T03:54:30.000000Z","id":1},"message":"created"
}
```

Step 10 — GET enrollment by ID (enriched) (Expected: HTTP 200)

```
curl -i http://localhost:8003/api/enrollments/1
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i http://localhost:8003/api/enrollments/1
HTTP/1.1 200 OK
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:54:45 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"data":{"id":1,"student":{"id":4,"name":"Sequential Student Updated","email":"seq.student.updated@example.com","created_at":"2026-03-15T03:52:23.00000
0Z","updated_at":"2026-03-15T03:53:07.000000Z"},"course":{"id":4,"title":"Sequential Course Updated","description":"Updated for sequential test","creat
ed_at":"2026-03-15T03:53:25.000000Z","updated_at":"2026-03-15T03:54:08.000000Z"},"enrolled_at":"2026-03-15T03:54:30.000000Z"},"message":"success"}
```

Step 11 — GET enrollments by student (Expected: HTTP 200)

```
curl -i http://localhost:8003/api/enrollments/student/4
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i http://localhost:8003/api/enrollments/student/4
HTTP/1.1 200 OK
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:55:07 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 57
Access-Control-Allow-Origin: *

{"data":[{"id":1,"student":{"id":4,"name":"Sequential Student Updated","email":"seq.student.updated@example.com","created_at":"2026-03-15T03:52:23.0000
00Z","updated_at":"2026-03-15T03:53:07.000000Z"},"course":{"id":4,"title":"Sequential Course Updated","description":"Updated for sequential test","crea
ted_at":"2026-03-15T03:53:25.000000Z","updated_at":"2026-03-15T03:54:08.000000Z"},"enrolled_at":"2026-03-15T03:54:30.000000Z"}],"message":"success"}
```

Step 12 — Validation — POST student missing name (Expected: HTTP 400)

```
curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"email":"missing-name@example.com"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"email":"missing-name@example.com"}'
HTTP/1.1 400 Bad Request
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:56:07 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"VALIDATION_ERROR","message":"The name field is required."}
```

Step 13 — Validation — POST student invalid email (Expected: HTTP 400)

```
curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Bad Email","email":"not-an-email"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Bad Email","email":"not-an-email"}'
HTTP/1.1 400 Bad Request
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:56:55 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"VALIDATION_ERROR","message":"The email field must be a valid email address."}
```

Step 14 — Validation — POST enrollment missing course_id (Expected: HTTP 400)

```
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4}'
HTTP/1.1 400 Bad Request
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:57:16 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"VALIDATION_ERROR","message":"The course id field is required."}
```

Step 15 — Not Found — GET nonexistent student (Expected: HTTP 404)

```
curl -i http://localhost:8001/api/students/9999
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i http://localhost:8001/api/students/9999
HTTP/1.1 404 Not Found
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:58:19 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Student with id 9999 does not exist"}
```

Step 16 — Not Found — GET nonexistent course (Expected: HTTP 404)

```
curl -i http://localhost:8002/api/courses/9999
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i http://localhost:8002/api/courses/9999
HTTP/1.1 404 Not Found
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:58:35 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Course with id 9999 does not exist"}
```

Step 17 — **Not Found — GET nonexistent enrollment** (Expected: HTTP 404)

```
curl -i http://localhost:8003/api/enrollments/9999
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i http://localhost:8003/api/enrollments/9999
HTTP/1.1 404 Not Found
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:58:53 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Enrollment with id 9999 does not exist"}
```

Step 18 — **Duplicate — POST student existing email** (Expected: HTTP 409)

```
curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Duplicate Student","email":"juan@example.com"}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X POST http://localhost:8001/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Duplicate Student","email":"juan@example.com"}'
HTTP/1.1 409 Conflict
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:59:15 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"DUPLICATE_EMAIL","message":"A student with this email already exists"}
```

Step 19 — **Duplicate — POST duplicate enrollment pair** (Expected: HTTP 409)

```
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
>   -H "Content-Type: application/json" \
>   -d '{"student_id":4,"course_id":4}'
HTTP/1.1 409 Conflict
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:59:33 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"DUPLICATE_ENROLLMENT","message":"This student is already enrolled in this course"}
```

Step 20 — Cross-service 404 — nonexistent student_id (Expected: HTTP 404)

```
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":9999,"course_id":4}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":9999,"course_id":4}'
HTTP/1.1 404 Not Found
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 03:59:57 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"STUDENT_NOT_FOUND","message":"Student with id 9999 does not exist"}
```

Step 21 — Cross-service 404 — nonexistent course_id (Expected: HTTP 404)

```
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":9999}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":9999}'
HTTP/1.1 404 Not Found
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:00:15 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"COURSE_NOT_FOUND","message":"Course with id 9999 does not exist"}
```

Step 22 — Dependency Down — student-service offline (Expected: HTTP 503)

MANUAL STEP REQUIRED

1. Press Ctrl+C on Terminal 1 to stop student-service (port 8001)
2. Run the curl command below in Terminal 4
3. Restart student-service on Terminal 1 before Step 23

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
HTTP/1.1 503 Service Unavailable
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:01:25 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"SERVICE_UNAVAILABLE","message":"A dependency service is not responding"}
```

```
# Stop student-service first (Ctrl+C on Terminal 1)
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
```

Step 23 — Dependency Timeout — student-service too slow (Expected: HTTP 504)

MANUAL STEP REQUIRED

1. Restart student-service if stopped from Step 22
2. Open: lab2/services/student-service/app/Http/Controllers/StudentController.php
3. In the show() method, add sleep(10); as the first line
4. Restart student-service on Terminal 1
5. Run the curl command below — it will timeout after 5 seconds
6. REMOVE sleep(10) immediately after capturing the screenshot

```
# Add sleep(10) to StudentController::show() first, then restart student-service
curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X POST http://localhost:8003/api/enrollments \
  -H "Content-Type: application/json" \
  -d '{"student_id":4,"course_id":4}'
HTTP/1.1 504 Gateway Timeout
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:05:44 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"GATEWAY_TIMEOUT","message":"Dependency service took too long to respond"}
```

Step 24 — Cleanup — DELETE enrollment (Expected: HTTP 200)

```
curl -i -X DELETE http://localhost:8003/api/enrollments/1
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i -X DELETE http://localhost:8003/api/enrollments/1
HTTP/1.1 200 OK
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:06:29 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":null,"message":"deleted"}
```

Step 25 — Cleanup — DELETE student (Expected: HTTP 200)

```
curl -i -X DELETE http://localhost:8001/api/students/4
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i -X DELETE http://localhost:8001/api/students/4
HTTP/1.1 200 OK
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:06:46 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":null,"message":"deleted"}
```

Step 26 — Cleanup — DELETE course (Expected: HTTP 200)

```
curl -i -X DELETE http://localhost:8002/api/courses/4
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i -X DELETE http://localhost:8002/api/courses/4
HTTP/1.1 200 OK
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:07:09 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"data":null,"message":"deleted"}
```

Step 27 — Verify cleanup — all three deleted resources return 404 (Expected: HTTP 404)

```
curl -i http://localhost:8001/api/students/4

curl -i http://localhost:8002/api/courses/4

curl -i http://localhost:8003/api/enrollments/1
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/course-service (main)
$ curl -i http://localhost:8002/api/courses/4
HTTP/1.1 404 Not Found
Host: localhost:8002
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:08:29 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Course with id 4 does not exist"}
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/student-service (main)
$ curl -i http://localhost:8001/api/students/4
HTTP/1.1 404 Not Found
Host: localhost:8001
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:08:12 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Student with id 4 does not exist"}
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab2/services/enrollment-service (main)
$ curl -i http://localhost:8003/api/enrollments/1
HTTP/1.1 404 Not Found
Host: localhost:8003
Connection: close
X-Powered-By: PHP/8.4.0
Cache-Control: no-cache, private
Date: Sun, 15 Mar 2026 04:08:49 GMT
Content-Type: application/json
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 58
Access-Control-Allow-Origin: *

{"error":"NOT_FOUND","message":"Enrollment with id 1 does not exist"}
```

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

All 27 test steps executed in sequential order returned the correct HTTP status codes. Steps 1 through 11 confirmed that all three services respond correctly to valid CRUD operations, including enriched responses from enrollment-service that embed student and course data. Steps 12 through 14 confirmed that validation errors return HTTP 400 with VALIDATION_ERROR and field-level error messages. Steps 15 through 17 confirmed that not-found errors return HTTP 404 with NOT_FOUND for direct lookups. Steps 20 and 21 confirmed cross-service not-found handling: enrollment-service correctly maps upstream 404 responses to STUDENT_NOT_FOUND and COURSE_NOT_FOUND.

Duplicate detection in Steps 18 and 19 worked correctly. Student-service returned DUPLICATE_EMAIL when the same email was submitted twice. Enrollment-service returned DUPLICATE_ENROLLMENT when the same student–course pair was submitted twice. The dependency failure tests in Steps 22 and 23 confirmed that enrollment-service distinguishes between an offline dependency and a slow dependency. A stopped service triggers ConnectionException without 'timed out' in the message, returning HTTP 503. A service with sleep(10) exceeds the Http::timeout(5) threshold and triggers ConnectionException with 'timed out', returning HTTP 504. Steps 24 through 27 confirmed that all three delete operations succeed and that deleted resources return 404 when queried.

IV. LAB DISCUSSIONS / COMPUTATIONS

Consistent ERROR_CODE strings across all three services are essential for frontend error handling. The Academe frontend uses a single error handler that reads the 'error' field from every JSON response. Because all three services follow the same {"error": "...", "message": "..."} response contract, the frontend does not require service-specific error logic. This consistency was confirmed across all 27 test steps — no service returned an unexpected format or an undocumented error code.

Only the enrollment-service returns 503 and 504 errors because it is the only service with external HTTP dependencies. When creating an enrollment, the controller makes two sequential HTTP GET calls — one to student-service to verify the student exists and one to course-service to verify the course exists. Student-service and course-service query only their own local SQLite databases and have no outbound calls, making 503 and 504 responses architecturally impossible for those services.

The Http::timeout(5) configuration protects the system from hanging indefinitely when a dependency is unresponsive. The serviceError() helper method inspects the ConnectionException message using str_contains($e->getMessage(), 'timed out') to distinguish the two failure modes. This is a functional approach for this lab scope. In production, this would be replaced by circuit breakers with explicit /health endpoints to short-circuit requests to a known-failing service without waiting for the timeout.

Using curl -i rather than plain curl was essential for this test suite. Without the -i flag, curl only displays the response body and the HTTP status line (e.g., HTTP/1.1 503 Service Unavailable) is invisible. Since the entire purpose of edge case testing is verifying correct HTTP status codes, every command in the suite uses -i. The sequential ordering of steps was equally critical — Step 9 depends on IDs created in Steps 3 and 6, and Step 19 depends on the enrollment created in Step 9. Without deterministic ordering, data-dependency failures would obscure real bugs.

V. CONCLUSIONS

All 27 sequential test steps were executed and confirmed to return correct HTTP status codes and JSON error bodies. The test suite covered every endpoint across all three microservices: happy path CRUD (Steps 1–11), validation errors (Steps 12–14), not-found scenarios (Steps 15–17 and 20–21), duplicate detection (Steps 18–19), dependency failures (Steps 22–23), and cleanup verification (Steps 24–27). Microservices introduce failure modes — 503 Service Unavailable and 504 Gateway Timeout — that monolithic systems do not face because monoliths have no external HTTP dependencies. The enrollment-service is the single point of cross-service communication and therefore the only service requiring resilience handling. Production improvements would add circuit breakers to prevent cascade failures, retry logic with exponential backoff for transient errors, and /health endpoints on each service for monitoring and load balancer integration.

References

Laravel Documentation. (2024). The PHP Framework for Web Artisans. https://laravel.com/docs

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Richardson, C. (2018). Microservices Patterns. Manning Publications.

Mozilla Developer Network. (2024). HTTP Status Codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

curl Documentation. (2024). curl — command line tool and library. https://curl.se/docs/